

DSA PATTERN RECOGNITION

From Problem Statement → Correct Data Structure & Algorithm

This guide fixes the exact gap most people hit after learning DSA: knowing every algorithm individually but freezing when a fresh problem shows up, because you can't tell **which one applies**. Every entry below follows one format: the **signal** (what you literally see/read in the problem), the **pattern** to reach for, **why** it fits, and 2-3 **real problems** to drill it on (mostly LeetCode).

How to use this in an interview / contest: Read the problem once for the story. Then re-read it hunting only for the signal words in bold in each table ("sorted", "subarray", "k largest", "pairs", "shortest path", "count ways"...). Match the signal → look up the pattern → recall the template you've already drilled. Pattern-matching beats reinventing the algorithm from scratch every time.

Prepared for Bhavesh — AI & DS, GSV Vadodara. Companion to the DSA/DAA sprint plan.

0. THE 60-SECOND DECISION FLOW

Before diving into patterns, run through this checklist top to bottom the moment you finish reading a problem. Stop at the first question that fits — it usually narrows you to 1-2 candidate patterns.

Ask yourself...	If YES → likely pattern
Is the array/string already sorted, or can I sort it cheaply?	Binary Search, Two Pointers
Do I need a contiguous subarray/substring satisfying a condition?	Sliding Window
Do I need pairs/triplets that sum to something, from a sorted array?	Two Pointers
Do I need O(1) lookup of "have I seen this before"?	Hash Map / Hash Set
Do I repeatedly need the min/max/kth element from a changing set?	Heap (Priority Queue)
Is it a tree/graph and I need shortest path or level order?	BFS
Is it a tree/graph and I need to explore all paths / connectivity?	DFS / Union-Find
Do choices depend on earlier choices, with overlapping subproblems?	Dynamic Programming
Do I need every combination/permutation/subset, pruning invalid ones?	Backtracking
Do I need next-greater/previous-smaller or to match brackets?	Monotonic Stack
Do I need the running max/min in a sliding window efficiently?	Monotonic Deque
Am I comparing prefix strings / doing autocomplete?	Trie
Does a greedy local choice provably lead to a global optimum?	Greedy (sort + scan)
Are values bounded/small integers, or do I need XOR tricks?	Bit Manipulation

1. TWO POINTERS

Use when you can eliminate possibilities by moving from both ends (or two speeds) instead of checking every pair → turns $O(n^2)$ into $O(n)$.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
Array is sorted + need pair/triplet with a target sum	Two Pointers (opposite ends, move based on sum vs target)	Sorted order lets you discard a whole side in $O(1)$ instead of rechecking	Two Sum II, 3Sum, 3Sum Closest
"Remove duplicates in-place", "partition array"	Two Pointers (slow/fast, same direction)	One pointer writes the answer position, other scans	Remove Duplicates from Sorted Array, Move Zeroes
"Container with most water", max area between two lines	Two Pointers (greedy shrink from wider side)	Moving the shorter boundary is the only move that can improve area	Container With Most Water, Trapping Rain Water
Detect a cycle in a linked list / find middle element	Fast & Slow Pointers (Floyd's)	Fast pointer laps slow pointer iff a cycle exists; slow reaches middle when fast reaches end	Linked List Cycle II, Middle of Linked List, Happy Number
Compare/merge two sorted arrays or strings	Two Pointers (one per array)	Both sequences are sorted, so a single forward pass merges them	Merge Sorted Array, Is Subsequence

2. SLIDING WINDOW

Use when the problem talks about a **contiguous** block of an array/string and asks for an optimum (longest/shortest/max sum) under a condition. Avoids recomputation by growing/shrinking one window.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Longest/shortest substring with at most K distinct chars"	Variable-size sliding window + hash map of counts	Expand right to include, shrink left when constraint breaks, track best window	Longest Substring Without Repeating Chars, Longest Substring with K Distinct
"Max sum subarray of size K" (fixed size)	Fixed-size sliding window	Add new element, remove element leaving the window — $O(1)$ per step instead of resumming	Maximum Average Subarray I, Sliding Window Maximum
"Minimum window containing all characters of T"	Variable window + frequency map + "have/need" counters	Grow until valid, then greedily shrink to find the minimal valid window	Minimum Window Substring, Permutation in String
Need the running max/min inside every window efficiently	Monotonic Deque + sliding window	Deque keeps candidates in decreasing/increasing order so front is always the answer	Sliding Window Maximum, Shortest Subarray with Sum $\geq K$

3. HASHING (HashMap / HashSet)

Default reflex whenever you need **O(1) membership/frequency lookup** and order doesn't matter. If you catch yourself writing a nested loop just to "check if X exists", stop — hash it instead.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Have I seen this value/pair/state before?"	HashSet for existence checks	O(1) average lookup instead of O(n) linear scan	Two Sum, Contains Duplicate, Longest Consecutive Sequence
Need counts/frequency of elements (anagrams, majority, etc.)	HashMap<value, count>	Single pass builds frequency; second pass answers the query	Valid Anagram, Group Anagrams, Top K Frequent Elements
"Subarray sum equals K" / "count subarrays with sum X"	Prefix Sum + HashMap<prefixSum, count>	$\text{sum}(i,j) = \text{prefix}[j] - \text{prefix}[i-1]$; store prefix sums seen so far to find matches in O(1)	Subarray Sum Equals K, Continuous Subarray Sum
Need O(1) lookup AND O(1) insert order / LRU behaviour	HashMap + Doubly Linked List	HashMap gives O(1) access to nodes; linked list gives O(1) reordering/eviction	LRU Cache, LFU Cache

4. BINARY SEARCH

Not just "find X in sorted array". The real trigger is: **can I turn the answer space into a monotonic yes/no function I can binary search over?** This is the most under-used pattern by people at your stage.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
Array is sorted, find an element / insertion point	Classic Binary Search	Halve the search space every step $\rightarrow O(\log n)$	Binary Search, Search Insert Position
"Minimize the maximum" or "maximize the minimum" of some value	Binary Search on the Answer	If value V works, every value $>V$ (or $<V$) also works — monotonic, so binary search the feasibility function	Koko Eating Bananas, Capacity To Ship Packages, Split Array Largest Sum
Sorted array is rotated, need to find element/min	Modified Binary Search (check which half is sorted)	One half is always properly sorted; decide which half to keep using that	Search in Rotated Sorted Array, Find Minimum in Rotated Sorted Array
Need k-th smallest/largest across multiple sorted lists/matrix	Binary Search on value range	Binary search a candidate value, count how many elements $\leq$ it using the sorted structure	Kth Smallest Element in Sorted Matrix, Median of Two Sorted Arrays

5. STACK

Reflex trigger: the problem cares about the **most recent unmatched/unresolved thing** — nesting, matching, or "next greater/smaller" relationships.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
Parentheses/brackets/tags matching or validity	Stack (push open, pop & match on close)	LIFO naturally mirrors nested structure — last opened must be first closed	Valid Parentheses, Generate Parentheses, Remove Invalid Parentheses
"Next greater/smaller element" for every array element	Monotonic Stack (increasing or decreasing)	Pop elements that can never be the answer for future indices — each element pushed/popped once = $O(n)$	Next Greater Element I/II, Daily Temperatures, Largest Rectangle in Histogram
Evaluate expressions (postfix, calculator, nested)	Stack for operators/operands	Defers evaluation until you have both operands / closing symbol	Evaluate RPN, Basic Calculator, Decode String
Need to undo the most recent operation / simulate recursion iteratively	Explicit Stack (mimics call stack)	Converts recursive DFS into iterative form, avoids stack overflow on deep recursion	Iterative Inorder/Preorder Traversal, Flatten Nested List

6. QUEUE / DEQUE

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
Process items "in the order they arrived", level by level	Queue (FIFO)	Guarantees you fully process one level/layer before the next	Binary Tree Level Order Traversal, Rotting Oranges
Need max/min of every sliding window in $O(n)$ total	Monotonic Deque	Front of deque is always current window's max (or min); each element added/removed once	Sliding Window Maximum
Simulate a task scheduler / round robin with cooldowns	Queue + HashMap for cooldown tracking	Order of arrival matters, and you need $O(1)$ recency checks alongside it	Task Scheduler, Design Hit Counter

7. TREES

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Level order", "shortest depth", "width of tree"	BFS with a Queue	BFS naturally processes tree level by level	Level Order Traversal, Minimum Depth of Binary Tree
"Path sum", "max depth", "diameter", explore every path	DFS (recursive, pre/in/post-order)	Recursion mirrors the tree's recursive structure; post-order lets children answer before parent decides	Path Sum II, Diameter of Binary Tree, Max Depth
Binary Search Tree operations (search/insert/kth smallest)	BST property (left < node < right) + inorder traversal	Inorder traversal of a BST yields sorted order for free	Validate BST, Kth Smallest in BST, BST Iterator
"Lowest common ancestor"	DFS returning found-node up the recursion	Post-order return values let a parent detect when both targets were found in different subtrees	LCA of Binary Tree, LCA of BST
Need to serialize/reconstruct a tree, or answer many range queries on it	Segment Tree / Fenwick (BIT) for range query+update	$O(\log n)$ update and $O(\log n)$ range query instead of $O(n)$ rebuild	Range Sum Query - Mutable, Count of Smaller Numbers After Self
Prefix-based string search (autocomplete, word search)	Trie (Prefix Tree)	Shares common prefixes across words $\rightarrow O(\text{word length})$ lookup instead of scanning all words	Implement Trie, Word Search II, Design Add and Search Words

8. GRAPHS

Graph questions rarely say "graph" — look for: grid, network, dependency, connection, rooms, islands, courses with prerequisites. Model it as nodes+edges first, pick algorithm second.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
Shortest path, unweighted graph or grid	BFS	BFS explores in increasing distance order, so first time you reach target = shortest path	Word Ladder, Rotting Oranges, Shortest Path in Binary Matrix
Shortest path, weighted graph, non-negative weights	Dijkstra's Algorithm (Min-Heap)	Greedily finalize the closest unvisited node; heap gives $O(\log n)$ extraction of the min	Network Delay Time, Cheapest Flights Within K Stops, Path With Minimum Effort
Weighted graph with possible negative edges	Bellman-Ford	Relaxes all edges $V-1$ times, tolerant of negative weights, detects negative cycles	Cheapest Flights (K stops variant), Negative Cycle Detection
"Count islands", "connected components", flood fill	DFS or BFS traversal marking visited cells/nodes	Every connected region gets fully explored and marked exactly once	Number of Islands, Flood Fill, Max Area of Island
"Course schedule", task ordering with dependencies	Topological Sort (Kahn's BFS or DFS + stack)	Only valid if graph is a DAG; order respects all "must come before" edges	Course Schedule I/II, Alien Dictionary
"Are these two nodes connected?", dynamically merging groups	Union-Find (Disjoint Set Union)	Near $O(1)$ amortized union/find with path compression + union by rank — avoids re-running DFS repeatedly	Number of Provinces, Redundant Connection, Accounts Merge
Minimum cost to connect all nodes (spanning tree)	Prim's or Kruskal's (MST) — Kruskal uses Union-Find	MST connects all nodes with minimum total edge weight, no cycles	Min Cost to Connect All Points, Connecting Cities With Minimum Cost
Grid where you can move in all 8/4 directions to find shortest steps	BFS on implicit grid graph	Grid cells = nodes, adjacency = valid moves; BFS gives shortest step count	Shortest Path in Binary Matrix, Sliding Puzzle

9. HEAP / PRIORITY QUEUE

Trigger words: "**kth largest/smallest**", "**top K**", "**merge K sorted**", or anything where you repeatedly need the current min/max from a changing collection.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Kth largest/smallest element" (static or streaming)	Min-Heap of size K (for kth largest) / Max-Heap of size K (for kth smallest)	Keep only K candidates $\rightarrow O(n \log k)$ instead of $O(n \log n)$ full sort	Kth Largest Element in Array, Kth Largest Element in a Stream
"Top K frequent", "K closest points"	Heap keyed by frequency/distance	Same idea — heap maintains the best K seen so far without a full sort	Top K Frequent Elements, K Closest Points to Origin
Merge K sorted lists/arrays	Min-Heap holding one "current head" from each list	Always pop the global minimum among K heads in $O(\log K)$ instead of comparing all K each time	Merge K Sorted Lists
Need running median from a stream	Two Heaps (max-heap for lower half, min-heap for upper half)	Keeps both halves balanced; median is always at the top of one/both heaps	Find Median from Data Stream
Greedy scheduling by "earliest deadline" or "least frequent gap"	Heap-driven greedy	Always process the currently most urgent/least-used item next	Task Scheduler, Meeting Rooms II

10. DYNAMIC PROGRAMMING

DP applies when a problem has **overlapping subproblems** + **optimal substructure** (the answer to the whole depends on answers to smaller versions of itself). If you can write a brute-force recursion and you notice the same (state) being recomputed — that's your confirmation. Below are the sub-patterns, which matter more than the umbrella term "DP".

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Number of ways to reach step N / climb / tile"	1D DP, $dp[i]$ built from $dp[i-1]$, $dp[i-2]$...	Current state depends only on a fixed small window of previous states	Climbing Stairs, House Robber, Decode Ways
"Longest increasing/common subsequence"	1D or 2D DP over sequence positions	$dp[i]$ = best answer ending at/using index i , built from earlier dp values	Longest Increasing Subsequence, Longest Common Subsequence
"Can I partition/select items to hit an exact sum/capacity" (each item used once)	0/1 Knapsack DP, $dp[i][capacity]$	Each item has a binary include/exclude choice interacting with a capacity constraint	Partition Equal Subset Sum, Target Sum, 0/1 Knapsack
Items can be reused unlimited times to hit a target	Unbounded Knapsack DP	Same capacity-DP idea but the item choice loop doesn't exclude reuse	Coin Change, Coin Change II, Rod Cutting
Two strings/sequences being compared/transformed into each other	2D DP, $dp[i][j]$ over both strings	State needs position in both sequences simultaneously	Edit Distance, Longest Common Subsequence, Interleaving String
"Min/max cost path through grid" with move restrictions	2D Grid DP	$dp[r][c]$ depends on a small fixed set of neighbor cells (up/left etc.)	Minimum Path Sum, Unique Paths, Dungeon Game
"Best way to cut/partition a sequence into parts" (matrix chain style)	Interval DP, $dp[i][j]$ over subarray $[i, j]$	Optimal split point for a range depends on optimal splits of sub-ranges within it	Matrix Chain Multiplication, Burst Balloons, Palindrome Partitioning II
State also depends on some extra constraint ("at most K transactions", "holding stock or not")	State-Machine / Multi-dimension DP	Add an explicit dimension to dp for that extra constraint/state	Best Time to Buy/Sell Stock III/IV, House Robber II
Need DP over subsets of a small set ($n \leq \sim 20$)	Bitmask DP, $dp[mask][i]$	Represent "which items used" as bits $\rightarrow 2^n$ states is feasible for small n	Travelling Salesman (DP), Partition to K Equal Sum Subsets

11. BACKTRACKING

Trigger words: **"all possible"**, **"every combination/permutation/subset"**, **"generate all valid ..."**. DP optimizes a single best answer; backtracking enumerates all answers, pruning branches early when they can't work.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"All subsets / power set"	Backtracking: include/exclude each element	Binary choice tree over elements, depth = n	Subsets, Subsets II
"All permutations"	Backtracking with a used[] / visited tracker	Try every unused element at each position, undo choice on backtrack	Permutations, Permutations II
"All combinations summing to target"	Backtracking + pruning when running sum exceeds target	Prune early — don't explore branches that can't possibly reach target	Combination Sum I/II, Combination Sum III
Place items on a board under constraints (N-Queens, Sudoku)	Backtracking + constraint check before placing	Check validity before recursing deeper — avoids wasted exploration	N-Queens, Sudoku Solver, Word Search

12. GREEDY

Use only when you can **prove** a locally optimal choice never hurts the global optimum (usually via an exchange argument). If you're unsure whether greedy actually works, it probably doesn't — default back to DP.

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Maximum non-overlapping intervals", scheduling	Sort by end time, greedily pick	Picking the interval that ends earliest always leaves maximum room for future picks	Non-overlapping Intervals, Activity Selection
"Minimum number of platforms/rooms needed"	Sort start & end times separately, sweep	Track concurrent overlaps by sweeping sorted boundary events	Meeting Rooms II, Minimum Platforms
"Assign/distribute to maximize satisfied count" (gas station, candy, jump game)	Greedy single pass with local rule	Each local decision is provably safe/optimal, no need to look back	Gas Station, Jump Game II, Candy
Build optimal prefix-free binary codes	Huffman Coding (Greedy + Min-Heap)	Always merge the two least-frequent nodes first	Huffman Encoding (classic, not always on LeetCode)

13. INTERVALS

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Merge overlapping intervals"	Sort by start time, then linear merge	After sorting by start, overlap can only happen with the immediately previous interval	Merge Intervals, Insert Interval
"Find if a new interval conflicts" repeatedly	Sorted structure (TreeMap / sorted list) + binary search	$O(\log n)$ insert/lookup instead of scanning all intervals each time	My Calendar I/II/III

14. BIT MANIPULATION

Signal in the problem	Pattern / Data Structure	Why it fits	Practice
"Find the single/unique number among duplicates"	XOR trick	$a \oplus a = 0$, so all pairs cancel out, leaving only the unique element	Single Number I/II/III
"Count set bits", power-of-two checks	Bit tricks: $n \& (n-1)$, $n \& (-n)$	$n \& (n-1)$ clears the lowest set bit → useful for counting bits or checking power of 2	Number of 1 Bits, Power of Two
Subsets of a small set need to be enumerated fast	Bitmask enumeration (0 to $2^n - 1$)	Every integer in that range represents exactly one subset via its bit pattern	Subsets (bitmask approach), Max AND Sum of Array

15. TIME COMPLEXITY QUICK REFERENCE

Use this to sanity-check whether your chosen pattern actually fits the given constraints (a huge tell in interviews: constraints hint at the required complexity).

Constraint (n =)	Expected complexity	Typical pattern family
$n \leq \sim 10$	$O(2^{\sup 2n})$ or $O(n!)$	Backtracking, brute force permutations
$n \leq \sim 20$	$O(2^n)$	Bitmask DP, subset enumeration
$n \leq \sim 500$	$O(n^2)$ or $O(n^3)$	2D DP, Floyd-Warshall, brute pairwise
$n \leq \sim 10^4 - 10^5$	$O(n \log n)$	Sorting, Binary Search, Heap, Divide & Conquer
$n \leq \sim 10^6+$	$O(n)$ or $O(n \log n)$	Two Pointers, Sliding Window, Hashing, single-pass DP
Very large / streaming	$O(\log n)$ or $O(1)$ per operation	Heap ops, Union-Find, Segment Tree, Hashing

Practical tip: when you're stuck mid-problem, don't ask "which algorithm is this" — ask "what are the constraints telling me about complexity", then "what operation do I need repeatedly (lookup / min-max / range-query / connectivity)", then match that operation to the tables above. That two-step habit is what separates people who "know DSA" from people who can actually apply it live.